



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto: Sistema Verificador de Cumplimiento
OWASP**

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

- Andia Navarro, Diego Fabrizio (2022073906)
- Concha Llaca, Gerardo Alejandro (2017057849)

Tacna – Perú

2026

Presentado por:

Concha Llaca, Gerardo Alejandro / Andia Navarro, Diego Fabrizio

Estudiantes de la Escuela Profesional de Ingeniería de Sistemas – UPT

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Andia Navarro, D. / Concha Llaca, G.	Mag. Ing. Patrick Cuadros Quiroga	Mag. Ing. Patrick Cuadros Quiroga	04/07/2026	Versión Original

ÍNDICE

Resumen Ejecutivo	3
I. Propuesta narrativa	4
1. Planteamiento del Problema	4
2. Justificación del proyecto	4
3. Objetivo general	4
4. Beneficios	4
5. Alcance	5
6. Requerimientos del sistema	5
7. Restricciones	5
8. Supuestos	5
9. Resultados esperados	5
10. Metodología de implementación	5
11. Actores claves	6
12. Papel y responsabilidades del personal	6
13. Plan de monitoreo y evaluación	6
14. Cronograma del proyecto	6
15. Hitos de entregables	7
II. Presupuesto	8
1. Planteamiento de aplicación del presupuesto	8
2. Presupuesto	8
3. Análisis de Factibilidad	8
4. Evaluación Financiera	8
Anexo 01 – Requerimientos del Sistema Verificador de Cumplimiento OWASP	9
a) Requerimientos Funcionales	9
b) Requerimientos No Funcionales	9

Resumen Ejecutivo

Nombre del Proyecto propuesto:	Sistema Verificador de Cumplimiento OWASP (OWASP Verificator), Tacna, 2026
Propósito del Proyecto y Resultados esperados:	El propósito del proyecto es dotar a Securify Software Solutions S.A.C. y a su comunidad de desarrolladores de un ecosistema que permita evaluar en tiempo real el cumplimiento del estándar OWASP Top 10 en código, URLs y repositorios de GitHub. Los resultados esperados son: • Una extensión de VS Code funcional con dashboard interactivo de cumplimiento. • Un sistema web y API REST desplegados en la nube para auditoría de código y URLs. • Reducción medible del tiempo de detección de vulnerabilidades durante la codificación.
Población Objetivo:	Desarrolladores independientes, equipos de TI, auditores de seguridad y estudiantes/docentes de la Escuela Profesional de Ingeniería de Sistemas de la UPT.
Monto de Inversión (En Soles):	Duración del Proyecto (En Meses):
S/. 9,410.00	3 meses

I. Propuesta narrativa

1. Planteamiento del Problema

Los desarrolladores de software modernos enfrentan constantes ciberamenazas que vulneran la privacidad y estabilidad de los sistemas que construyen. A pesar de existir marcos de seguridad como el OWASP Top 10, la falta de retroalimentación inmediata durante la codificación provoca que se introduzcan vulnerabilidades de forma involuntaria: exposición de secretos y credenciales en texto plano, uso de funciones inseguras susceptibles a inyección (eval, exec), incorporación de dependencias desactualizadas con CVEs públicos y despliegue de endpoints sin cabeceras HTTP de protección (CSP, HSTS, X-Frame-Options).

Las herramientas SAST comerciales disponibles en el mercado (Checkmarx, Veracode, entre otras) resultan costosas y complejas de configurar para equipos pequeños, consultoras medianas o desarrolladores independientes como los de Securify Software Solutions S.A.C., lo que evidencia la necesidad de una alternativa ágil e integrada al flujo de trabajo del programador.

2. Justificación del proyecto

El proyecto se justifica por la necesidad de reducir el costo y el tiempo asociados a la detección de vulnerabilidades de seguridad en el ciclo de vida del software. Al integrar el análisis de seguridad directamente en el IDE (VS Code) y ofrecer una API centralizada de auditoría, se acerca la cultura DevSecOps a equipos que, por restricciones presupuestales, no pueden acceder a soluciones SAST comerciales. Adicionalmente, el proyecto constituye una oportunidad académica para aplicar conocimientos de calidad y pruebas de software en un caso real de la industria de ciberseguridad.

3. Objetivo general

Diseñar e implementar un sistema verificador que permita evaluar de manera ágil y en tiempo real el cumplimiento de los controles de seguridad OWASP Top 10 en código fuente, repositorios y URLs de producción, integrando interfaces web, una API REST y una extensión para Visual Studio Code.

4. Beneficios

Beneficios tangibles:

- Reducción de costos frente a licencias de herramientas SAST comerciales.
- Disminución del tiempo de corrección de vulnerabilidades detectadas tardíamente en producción.
- Ahorro de horas-hombre en auditorías manuales de cabeceras HTTP y dependencias.
- Beneficios intangibles:
- Fortalecimiento de la cultura de desarrollo seguro (DevSecOps) en los equipos de Securify Software Solutions S.A.C.
- Mayor confiabilidad y reputación frente a los clientes de la organización.
- Disponibilidad centralizada y trazable del histórico de auditorías de seguridad.

5. Alcance

El sistema (Web, CLI y Extensión de IDE) está dirigido, en su versión piloto, a Securify Software Solutions S.A.C. y sus equipos de desarrollo asociados. Incluye funcionalidades de escaneo estático de archivos, identificación de dependencias inseguras (CVEs), verificación de cabeceras HTTP de URLs y creación automática de issues en repositorios de GitHub. Estará disponible tanto para ejecución local (uso interactivo del desarrollador) como para acceso remoto vía API, siendo compatible con el editor Visual Studio Code y los principales navegadores web.

6. Requerimientos del sistema

Los requerimientos funcionales y no funcionales completos del sistema se detallan en el Anexo 01 – Requerimientos del Sistema del presente documento.

7. Restricciones

- El proyecto debe desarrollarse en un plazo máximo de 12 semanas, conforme al cronograma académico del curso de Calidad y Pruebas de Software.
- El equipo de desarrollo está conformado únicamente por dos integrantes.
- El motor de análisis estático se basa en expresiones regulares y no realiza análisis semántico profundo del código (no reemplaza un SAST comercial de nivel empresarial).
- El despliegue en producción depende de la disponibilidad del servicio en la nube (Azure App Service).

8. Supuestos

- Se asume que Securify Software Solutions S.A.C. proporcionará acceso a información representativa de sus procesos de desarrollo para efectos de validación del sistema.
- Se asume la disponibilidad continua de las APIs de terceros (GitHub, GitHub Copilot, Google Gemini) durante el periodo de desarrollo y pruebas.
- Se asume que los usuarios finales cuentan con Visual Studio Code instalado y con acceso a internet.

9. Resultados esperados

- Extensión de VS Code publicada en Visual Studio Code Marketplace y Open VSX Registry.
- Sistema web y API REST desplegados y accesibles públicamente.
- Reducción del tiempo de detección de vulnerabilidades OWASP Top 10 durante la codificación.
- Documentación técnica completa del sistema (visión, requerimientos, arquitectura y estándares de programación).

10. Metodología de implementación

El proyecto se desarrolla bajo un enfoque ágil adaptado al contexto académico, estructurado en las siguientes fases: (1) Levantamiento de información y visionamiento, (2) Especificación de requerimientos funcionales y no funcionales, (3) Diseño de la arquitectura de software en capas (presentación, negocio y datos), (4) Construcción incremental del backend (FastAPI), del frontend web y de la extensión de VS Code, (5) Pruebas unitarias, de integración y de comportamiento

(BDD con Behave/Pytest), y (6) Despliegue automatizado mediante GitHub Actions hacia Azure App Service.

11. Actores claves

Actor	Rol en el proyecto
Concha Llaca, Gerardo Alejandro	Analista-Desarrollador Full-Stack; responsable del backend (FastAPI), motor de análisis y despliegue.
Andia Navarro, Diego Fabrizio	Analista-Desarrollador Full-Stack; responsable de la extensión de VS Code, dashboard e integración con IA.
Mag. Ing. Patrick Cuadros Quiroga	Docente del curso; revisor y aprobador de los entregables del proyecto.
Securify Software Solutions S.A.C. (organización piloto)	Interesado principal; provee el contexto de negocio y valida los resultados del sistema.
Desarrolladores y auditores de seguridad	Usuarios finales del dashboard, la extensión de VS Code y la API.

12. Papel y responsabilidades del personal

Rol	Responsabilidades
Analista-Desarrollador Full-Stack (Backend)	Diseño de la API REST, modelado de datos (SQLite3/MySQL), implementación del motor de análisis estático y de escaneo de cabeceras HTTP, pruebas unitarias del backend.
Analista-Desarrollador Full-Stack (Extensión/Frontend)	Implementación de la extensión de VS Code, dashboard interactivo, integración con paneles de IA (Copilot/Gemini), pruebas de interfaz.
Docente del curso	Asesoría metodológica, revisión y aprobación de entregables (FD01 a FD08), control de calidad del proyecto.

13. Plan de monitoreo y evaluación

El avance del proyecto se monitorea mediante reuniones semanales de seguimiento entre los integrantes del equipo, el control de versiones en GitHub (historial de commits y ramas) y la ejecución continua de pruebas automatizadas (Pytest, Behave) en cada entrega. La evaluación de los entregables documentarios (FD01 a FD08) está a cargo del docente del curso, quien aprueba cada versión conforme a los criterios establecidos en el sílabo del curso de Calidad y Pruebas de Software. Adicionalmente, se evalúa el cumplimiento de los requerimientos no funcionales (rendimiento, seguridad, portabilidad) mediante pruebas de carga y auditorías internas antes de cada despliegue a producción.

14. Cronograma del proyecto

N.º	Actividad	Semanas
1	Levantamiento de información y elaboración del Documento de Visión (FD02)	1 – 2
2	Especificación de requerimientos (FD03) y estudio de factibilidad (FD01)	3 – 4
3	Diseño de la arquitectura de software (FD04)	5 – 6
4	Construcción del backend (API FastAPI) y motor de análisis estático	6 – 8
5	Construcción de la extensión de VS Code y del dashboard web	8 – 10

N.º	Actividad	Semanas
6	Pruebas unitarias, de integración y BDD	10 – 11
7	Despliegue en la nube, elaboración del informe final (FD05) y documentación complementaria (FD06-FD08)	11 – 12

15. Hitos de entregables

Hito	Entregable	Semana
H1	FD02 – Documento de Visión	2
H2	FD01 – Informe de Factibilidad / FD03 – Especificación de Requerimientos	4
H3	FD04 – Arquitectura de Software	6
H4	Versión funcional del backend y del motor de análisis estático	8
H5	Versión funcional de la extensión de VS Code y del dashboard	10
H6	Suite de pruebas automatizadas ejecutándose en CI/CD	11
H7	FD05 – Informe de Proyecto Final, FD06 – Propuesta de Proyecto, FD07 y FD08	12

II. Presupuesto

1. Planteamiento de aplicación del presupuesto

El presupuesto del proyecto se plantea desde una perspectiva académica, cubriendo únicamente los costos de personal, materiales generales, operación durante el desarrollo y ambiente tecnológico, dado que la mayoría de las herramientas de software empleadas (FastAPI, VS Code, GitHub, Pytest) son de licencia gratuita o de uso académico. En un escenario de adopción real por parte de Securify Software Solutions S.A.C., el presupuesto sería asumido íntegramente por la organización como parte de su inversión en herramientas internas de ciberseguridad.

2. Presupuesto

Concepto	Costo (S/.)
Costos generales (materiales de oficina)	260.00
Costos operativos durante el desarrollo	150.00
Costos del ambiente (hospedaje, dominio, licencias)	0.00
Costos de personal (2 desarrolladores x 12 semanas)	9,000.00
Presupuesto Total	9,410.00

3. Análisis de Factibilidad

El análisis de factibilidad técnica, económica, operativa, legal, social y ambiental del proyecto se desarrolla en detalle en el documento FD01 – Informe de Factibilidad. En síntesis, el proyecto resulta viable en todas sus dimensiones: técnicamente, por la disponibilidad y experiencia del equipo en las tecnologías propuestas; económicamente, por el bajo costo total de desarrollo (S/. 9,410.00) frente al ahorro estimado en licencias SAST comerciales; y operativa, legal, social y ambientalmente, por no presentar restricciones significativas.

4. Evaluación Financiera

Considerando un ahorro estimado anual de S/. 14,000.00 frente a licencias SAST comerciales y auditorías manuales, y un costo total de desarrollo de S/. 9,410.00, se obtiene una relación Beneficio/Costo (B/C) aproximada de 1.49, superior a la unidad, lo que valida la aceptación del proyecto. Asimismo, el Valor Actual Neto (VAN) proyectado a un año, con una tasa de descuento (COK) del 12%, resulta positivo, y la Tasa Interna de Retorno (TIR) estimada supera dicho costo de oportunidad, confirmando la viabilidad financiera del proyecto en el mediano plazo. El detalle completo de estos criterios se presenta en la sección 5 del documento FD01 – Informe de Factibilidad.

Anexo 01 – Requerimientos del Sistema Verificador de Cumplimiento OWASP

a) Requerimientos Funcionales

ID	Nombre del Requerimiento	Descripción	Prioridad
RF01	Escanear objetivos	El sistema recibe código, archivo local, URL web o URL de repositorio de GitHub para su análisis.	Alta
RF02	Identificar dependencias vulnerables	El sistema analiza las dependencias declaradas y las coteja contra una lista predefinida de CVEs.	Alta
RF03	Generar y validar tokens de API	El sistema expone endpoints seguros y gestiona tokens de acceso para clientes externos.	Alta
RF04	Escanear cabeceras HTTP	El sistema realiza una petición a una URL y evalúa la presencia y configuración de cabeceras de seguridad (CSP, HSTS, X-Frame-Options).	Alta
RF05	Mostrar dashboard interactivo	El sistema presenta un panel visual con el detalle de hallazgos, permitiendo filtrar, buscar y navegar a la línea vulnerable del código.	Alta
RF06	Sugerir remediación con IA	El sistema envía la vulnerabilidad detectada a los paneles de chat de GitHub Copilot o Google Gemini para sugerir una corrección.	Media
RF07	Aplicar correcciones automáticas (Quick Fixes)	El sistema ofrece arreglos automáticos para patrones específicos (por ejemplo, reemplazo de secretos expuestos por variables de entorno).	Media
RF08	Registrar historial de escaneos	El sistema almacena de forma local (SQLite3) o remota (MySQL) el registro de cada análisis realizado.	Media
RF09	Crear issues en GitHub	El sistema puede generar automáticamente tickets/issues de seguridad en el repositorio analizado.	Baja

b) Requerimientos No Funcionales

ID	Requerimiento No Funcional	Descripción / Criterio de Medida
RNF01	Rendimiento	El tiempo de escaneo de código local debe ser menor a 5 segundos, y el de URLs, menor a 15 segundos.
RNF02	Seguridad de acceso	Los endpoints de auditoría y generación de tokens requieren la cabecera X-API-Key.
RNF03	Arquitectura desacoplada	Separación estricta entre presentación, negocio (servicios) y acceso a datos.
RNF04	Portabilidad	Debe poder ejecutarse en Windows, Linux y macOS sin requerir bases de datos pesadas (persistencia dual SQLite/MySQL).
RNF05	Usabilidad e interfaz	El panel del IDE debe emplear un estilo premium tipo glassmorphism y ser totalmente adaptativo (responsivo).
RNF06	Extensibilidad	El motor estático de escaneo basado en expresiones regulares debe apoyarse en un archivo modular de firmas extensible.

ID	Requerimiento No Funcional	Descripción / Criterio de Medida
RNF07	Disponibilidad	El sistema web debe mantenerse accesible en producción con un tiempo de actividad objetivo superior al 95%.